

# Design Decisions

Rationale behind the key engineering choices

---

PAUT Explainable-AI Digital Twin — weld defect inspection from Total Focusing Method (TFM) images.

This document records the locked design decisions and working conventions for the project, so the rationale behind the code is captured in one place. The full technical design is in docs/architecture.md.

---

## 1. What this project is

An Explainable-AI Digital Twin for weld inspection from Phased Array Ultrasonic Testing (PAUT) TFM images. The system: (1) detects and classifies weld defects, (2) characterizes each defect (size, orientation, severity) from its mask, (3) explains predictions with model-appropriate XAI, and (4) presents everything in a Digital Twin dashboard.

The core model is an **SCN-Attention U-Net**: a fixed Wavelet Scattering branch fused with an Attention U-Net. The scattering prior generalizes from limited, noisy ultrasonic data, while the attention U-Net gives the precise masks needed for defect characterization.

## 2. Dataset (current state)

- data/raw/porosity/ — 525 porosity TFM images (G...-named).
- data/raw/slag/ — 525 slag TFM images (GS...-named).
- Total **1050 images, balanced 525 / 525**.
- Grayscale exported image files with small/thin/faint defects on a near-black background.
- Stage-1 pseudo-label masks are generated by the pipeline (no manual masks yet).

## 3. Locked decisions (do not deviate)

1. **Single amplitude channel.** Every input (color or grayscale) is converted to one normalized

amplitude channel before the scattering network — the model never depends on color.

2. **Classification is 2-class: porosity vs slag** (not a 4-class version). No "normal" or "mixed"

classes (no training examples for them).

3. **Segmentation is 3-channel:** background, porosity, slag.

4. **"No defect present" is detected from an empty segmentation mask** (defect pixel-area below a

small threshold), not from a trained class.

5. **Class balance:** the set is 525/525 balanced; inverse-frequency class weighting is enabled

in the classifier loss as a safeguard.

6. **Split = 70/15/15 (train/val/test), stratified by class, fixed seed.** Code does the split.

7. **Split BEFORE augmenting; augment ONLY the training split.** Validation/test contain untouched

originals (prevents data leakage). Splits are also weld-group-aware (all views of one weld stay in one split).

8. **Physics-aware augmentation** applied to the training split only.

9. **Scattering branch via Kymatio**, fixed filters (no gradients), J=2-3, L=8, order=2. Not trained.

10. **Loss = 0.6·Dice-Focal(seg) + 0.3·classification + 0.1·boundary**, with a foreground-focused

Tversky term added so thin defects are not lost to the background-vs-defect pixel imbalance.

11. **Characterization is derived from the mask** (blobs / regionprops / PCA): length, width, area,

orientation, centroid, equivalent diameter, aspect ratio, severity.

- pixel\_to\_mm is unknown for now → default 1.0, measurements reported in PIXELS. It is a

parameter, so a real mm value plugs in later with no code change. Scale-free outputs (orientation, aspect ratio, eccentricity) are correct regardless.

- Acceptance/severity thresholds are tunable placeholders pending a standard (ISO 5817 / ASME).

12. **XAI**: Grad-CAM / Grad-CAM++ (Seg-Grad-CAM) on the last decoder conv for the segmentation

head; the model's CBAM/attention-gate maps as intrinsic explanations; faithfulness metrics (deletion/insertion + pointing-game → trust score). Defect **type** is decided by a scattering-feature classifier (the segmentation/neural-head type signal is weaker).

13. **Digital-twin health / pass-fail is driven by mask-derived defect presence and severity**, not

by a "normal" classification.

## 4. Compute workflow — develop local, train in the cloud

The development laptop is CPU-only, so the project is structured for two environments:

- **Local (CPU)**: writing/debugging code, preprocessing, small-subset runs, unit tests, and the

Streamlit dashboard. Runnable on CPU with tiny batches.

- **Google Colab / Kaggle (free GPU)**: the actual model training on all 1050 images.

Conventions that make moving between them painless:

- All code is GPU-aware but CPU-runnable: device is detected with

`torch.device("cuda" if torch.cuda.is_available() else "cpu");` never hard-coded.

- All paths and hyperparameters live in configs/ (YAML), so the same code runs in both places by

swapping a config.

- notebooks/train\_colab.ipynb clones/loads the repo, installs requirements, mounts data, and runs

training.

- A `--limit N / subset` flag supports fast local smoke-tests on ~10 images.

## 5. Repository layout

```
paut-digital-twin/
├─ data/{raw,processed}/      raw is read-only originals
├─ src/{data,models,characterize,xai,twin,dashboard}/
├─ src/pipeline.py           one-click: image -> preprocess -> model -> characterize -> xai -> twin
├─ configs/                  yaml hyperparameters
├─ notebooks/                experiments, train_colab.ipynb
├─ scripts/                  utilities (contamination check, separability, ablation, packaging)
├─ tests/
├─ docs/                     architecture.md + reference PDFs
├─ requirements.txt
└─ README.md
```

## 6. Rules that must not be broken

- Do not augment before splitting, or let augmented copies cross into val/test (data leakage).
- Do not train the scattering filters.
- Do not add "normal" or "mixed" classes.
- Do not hard-code hyperparameters or device calls — use `configs/` and device detection.
- Build module by module; verify each before moving on.